



# GateGuard



SISTEMA DE ACESSOS E PAGAMENTOS

**GUIA PARA DESENVOLVEDORES**

## **Manual de Implantação API GateGuard**

Login centralizado e validação automática de acesso e pagamentos

Versão 1.0 · Integração backend-to-backend

## O que esta integração entrega

O GateGuard valida, em uma única chamada, a identidade do usuário, a situação do sistema e suas regras financeiras. No GateGuard, cada código `ORG_XXXX` representa um **sistema apartado**. A `ORG_0000` é reservada ao ambiente administrativo.

Navegador



Backend do sistema



API GateGuard

**Regra principal:** a chave completa existe somente no backend do sistema integrado. Ela nunca deve aparecer em HTML, JavaScript público, aplicativo cliente, repositório, log ou resposta enviada ao navegador.

### Responsabilidades

| Componente           | Responsabilidade                                                                                               |
|----------------------|----------------------------------------------------------------------------------------------------------------|
| Interface do sistema | Coletar apenas login e senha e chamar seu próprio backend.                                                     |
| Backend do sistema   | Ler <code>API_GATEGUARD</code> , acrescentar a organização internamente e chamar o GateGuard por HTTPS.        |
| GateGuard            | Validar chave, usuário, status de acesso e situação financeira; devolver somente dados públicos e autorização. |

### Pré-requisitos

- Backend próprio publicado com HTTPS (Render, Railway, Cloud Run ou equivalente).
- Node.js 20 ou tecnologia equivalente com suporte a requisições HTTPS.
- Chave gerada no painel GateGuard e copiada no momento da geração.
- Frontend configurado para chamar apenas o backend do próprio sistema.

## Atribuições e responsabilidades atuais

O GateGuard é o provedor de identidade, controle de acesso e validação financeira. Ele não substitui o banco operacional do sistema integrado. No GateGuard, cada `ORG_xxxx` representa um sistema apartado.

| Área                        | Responsável atual | Atribuição                                                                                   |
|-----------------------------|-------------------|----------------------------------------------------------------------------------------------|
| Identidade e autenticação   | GateGuard         | Validar login e senha nos ambientes Firebase central e apartado do sistema.                  |
| Perfis e permissões         | GateGuard         | Manter perfil, cargo e situação ativa ou inativa dos acessos.                                |
| Situação financeira         | GateGuard         | Aplicar vencimentos, bloqueios e liberações relacionados ao pagamento.                       |
| Credencial de integração    | Compartilhada     | O GateGuard gera e valida a chave; o backend integrado guarda a chave completa como segredo. |
| Sessão do sistema integrado | Sistema integrado | Criar, proteger, renovar e encerrar sua própria sessão após a autorização.                   |
| Dados operacionais          | Sistema integrado | Administrar tarefas, projetos, clientes, arquivos, conteúdo e regras próprias do produto.    |
| Interface e disponibilidade | Sistema integrado | Manter frontend, backend, domínio, infraestrutura e experiência dos seus usuários.           |

**Limite de atuação:** o GateGuard decide se o usuário pode acessar com base em identidade, autorização e situação financeira. Depois da autorização, o tratamento dos dados e das operações internas é responsabilidade do sistema integrado.

### Dados que não devem ser enviados ao GateGuard

- Tarefas, projetos, documentos e arquivos do sistema.
- Dados de negócio sem relação com autenticação, acesso ou pagamento.
- Segredos internos, tokens de outros fornecedores ou bancos operacionais.

### Responsabilidade do desenvolvedor integrador

- Manter `API_GATEGUARD` exclusivamente no backend.
- Proteger o endpoint de login com HTTPS, limitação de tentativas e logs seguros.
- Não registrar senhas, chaves ou tokens.
- Criar uma sessão própria e aplicar autorização também nas rotas internas.

## Etapa 1

# Configuração segura no Render

No serviço de backend do sistema, acesse **Environment** e crie:

| Key           | Value                                                  |
|---------------|--------------------------------------------------------|
| API_GATEGUARD | {"id_org":"ORG_XXXX","KEYGG":"gg_live_CHAVE_COMPLETA"} |

Exemplo para o sistema identificado como ORG\_0001:

```
{"id_org":"ORG_0001","KEYGG":"gg_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"}
```

**Não há campo de organização no modal.** O backend obtém `id_org` dessa variável e acrescenta a informação à chamada interna.

## Variáveis auxiliares recomendadas

```
GATEGUARD_API_URL=https://logins-back.onrender.com
FRONTEND_URL=https://seu-dominio.example
SESSION_SECRET=SEGREDO_ALEATORIO_COM_32_OU MAIS CARACTERES
```

## Validação obrigatória ao iniciar

```
function gateGuardConfig() {
  let value;
  try { value = JSON.parse(process.env.API_GATEGUARD || ''); }
  catch { throw new Error('API_GATEGUARD contém JSON inválido.')}

  const organization = String(value.id_org || '').toUpperCase();
  const apiKey = String(value.KEYGG || '');
  if (!/^ORG_[A-Z0-9]{4,20}$/.test(organization))
    throw new Error('API_GATEGUARD.id_org inválido. ');
  if (!apiKey.startsWith('gg_live_') || apiKey.length < 40)
    throw new Error('API_GATEGUARD.KEYGG inválida. ');
  return { organization, apiKey };
}
```

## Etapa 2

# Endpoint no backend do sistema

O frontend chama POST /api/auth/login no backend do sistema. Esse endpoint injeta a organização e a chave e encaminha as credenciais ao GateGuard.

```
const express = require('express');
const { rateLimit } = require('express-rate-limit');

const app = express();
app.set('trust proxy', 1);
app.use(express.json({ limit: '5kb' }));

app.post('/api/auth/login', rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 10
}), async (req, res) => {
  const login = String(req.body?.login || '').trim().toLowerCase();
  const password = req.body?.password;
  if (!/^[a-z0-9._-]{2,80}$/i.test(login))
    return res.status(400).json({ error: 'Login inválido.' });
  if (typeof password !== 'string' || password.length < 6)
    return res.status(400).json({ error: 'Senha inválida.' });

  const { organization, apiKey } = gateGuardConfig();
  const response = await fetch(
    'https://logins-back.onrender.com/api/integrations/authenticate',
    {
      method: 'POST',
      headers: {
        'content-type': 'application/json',
        'x-gateguard-key': apiKey
      },
      body: JSON.stringify({ organization, login, password }),
      signal: AbortSignal.timeout(15000)
    }
  );

  const data = await response.json().catch(() => ({}));
  if (!response.ok)
    return res.status(response.status).json({
      error: data.error || 'Acesso não autorizado.'
    });

  // Crie aqui a sessão segura do próprio sistema.
  return res.json({
    authorized: true,
    organization: data.organization,
    user: data.user
  });
});
```

**Produção:** depois da autorização, crie uma sessão própria com cookie HttpOnly, Secure e SameSite adequado. Não use a senha nem a chave como token de sessão.

### Etapa 3

## Modal de login reutilizável

O botão “Entrar” abre um diálogo com apenas login e senha. Substitua `https://seu-backend.example` pelo backend do sistema.

```
<button type="button" id="gg-open-login">Entrar</button>

<dialog id="gg-login-modal" class="gg-modal">
  <form id="gg-login-form" class="gg-card">
    <button type="button" id="gg-close" class="gg-close"
      aria-label="Fechar">x</button>
    
    <h2>Acessar sistema</h2>
    <p>Informe suas credenciais de acesso.</p>
    <label>Login
      <input name="login" autocomplete="username" required>
    </label>
    <label>Senha
      <input name="password" type="password"
        autocomplete="current-password" required>
    </label>
    <button type="submit">Entrar</button>
    <p id="gg-feedback" role="alert"></p>
  </form>
</dialog>
```

```
const modal = document.querySelector('#gg-login-modal');
const form = document.querySelector('#gg-login-form');
const feedback = document.querySelector('#gg-feedback');

document.querySelector('#gg-open-login').onclick = () => modal.showModal();
document.querySelector('#gg-close').onclick = () => modal.close();

form.onsubmit = async event => {
  event.preventDefault();
  const button = form.querySelector('[type="submit"]');
  button.disabled = true;
  feedback.textContent = 'Validando acesso...';
  try {
    const response = await fetch(
      'https://seu-backend.example/api/auth/login',
      {
        method: 'POST',
        credentials: 'include',
        headers: { 'content-type': 'application/json' },
        body: JSON.stringify(Object.fromEntries(new FormData(form)))
      }
    );
    const data = await response.json().catch(() => ({}));
    if (!response.ok) throw new Error(data.error || 'Acesso negado.');
```

```
    location.href = '/painel';
  } catch (error) {
    feedback.textContent = error.message;
  } finally { button.disabled = false; }
};
```

## CSS sugerido para o modal

```
.gg-modal { border: 0; border-radius: 18px; padding: 0;
  box-shadow: 0 24px 80px #001a4433; }
.gg-modal::backdrop { background: #06162db8; backdrop-filter: blur(3px); }
.gg-card { position: relative; width: min(92vw, 390px); padding: 30px;
  display: grid; gap: 14px; font-family: Arial, sans-serif; }
.gg-card img { width: 240px; max-width: 85%; margin: 0 auto 8px; }
.gg-card h2,.gg-card p { margin: 0; text-align: center; }
.gg-card label { display: grid; gap: 6px; color: #344054;
  font-weight: 700; }
.gg-card input { height: 46px; border: 1px solid #cbd5e1;
  border-radius: 9px; padding: 0 12px; font: inherit; }
.gg-card button[type="submit"] { border: 0; border-radius: 9px;
  padding: 13px; background: #1677ff; color: white;
  font-weight: 800; cursor: pointer; }
.gg-close { position: absolute; right: 12px; top: 10px; border: 0;
  background: transparent; font-size: 24px; cursor: pointer; }
#gg-feedback { min-height: 20px; color: #b42318; font-size: 14px; }
```

### Experiência esperada

1. O usuário clica em “Entrar”.
2. O modal solicita somente login e senha.
3. O backend lê internamente a organização e a chave.
4. O GateGuard valida usuário, sistema, acesso e pagamento.
5. O sistema cria sua própria sessão e abre a área protegida.

**Resultado:** o usuário não precisa conhecer nem digitar `ORG_XXXX`; essa informação permanece interna e controlada pelo backend.

# Requisição e respostas

## Chamada exclusiva entre servidores

```
POST https://logins-back.onrender.com/api/integrations/authenticate
Content-Type: application/json
X-GateGuard-Key: gg_live_CHAVE_COMPLETA

{
  "organization": "ORG_0001",
  "login": "gestor",
  "password": "senha-do-usuario"
}
```

## Sucesso

```
{
  "authorized": true,
  "token": "token-temporario-da-integracao",
  "expiresIn": 300,
  "organization": { "id": "ORG_0001", "name": "Sistema" },
  "user": {
    "login": "gestor",
    "name": "Nome do usuário",
    "role": "gerente",
    "cargo": "Gestor"
  }
}
```

## Erros que devem ser apresentados sem detalhes técnicos

| HTTP | Significado                                       | Ação                                                  |
|------|---------------------------------------------------|-------------------------------------------------------|
| 400  | Login, senha ou organização inválidos.            | Revisar entrada/configuração.                         |
| 401  | Credenciais do usuário ou chave inválidas.        | Conferir login e os quatro finais da chave.           |
| 403  | Usuário desativado, sistema bloqueado ou vencido. | Orientar contato com o administrador.                 |
| 429  | Muitas tentativas.                                | Aguardar antes de repetir.                            |
| 5xx  | Serviço indisponível/configuração incompleta.     | Registrar apenas código e horário, nunca senha/chave. |



## Checklist antes de liberar

- ☐ API\_GATEGUARD está apenas no ambiente do backend.
- ☐ O `id_org` corresponde ao sistema fornecido pelo GateGuard.
- ☐ Os quatro últimos caracteres de `KEYGG` conferem com o painel GateGuard.
- ☐ O frontend não contém `gg_live_` em nenhum arquivo ou bundle.
- ☐ A chamada ao GateGuard acontece somente no backend e por HTTPS.
- ☐ CORS aceita somente os domínios oficiais do frontend.
- ☐ Há limite de tentativas no endpoint de login.
- ☐ Senha, chave e corpo de autenticação não aparecem em logs.
- ☐ Login válido e sistema regular liberam acesso.
- ☐ Senha incorreta retorna erro genérico.
- ☐ Usuário desativado não acessa.
- ☐ Sistema inativo ou vencido não acessa.
- ☐ Rotação da chave invalida a anterior e o Render recebe a nova.

## Teste rápido de saúde do backend integrado

Recomenda-se que cada backend ofereça GET `/api/health` sem revelar a chave:

```
{
  "status": "ok",
  "gateGuardConfigured": true,
  "organization": "ORG_0001"
}
```

Nunca devolva `KEYGG`, o JSON completo da variável ou o token temporário em endpoints de diagnóstico.

## Rotação, revogação e suporte

### Rotacionar uma chave

1. Gere/rotacione a chave no painel GateGuard.
2. Copie a nova chave imediatamente.
3. Atualize somente KEYGG dentro de API\_GATEGUARD no Render.
4. Salve e aguarde o redeploy/restart do backend.
5. Compare os quatro últimos caracteres com a prévia do GateGuard.
6. Execute um login de homologação.

### Revogação

Ao revogar a integração no GateGuard, novas autenticações param imediatamente. Remova também a variável do backend caso o sistema deixe de utilizar a integração.

### Informações seguras para suporte

- Código ORG\_XXXX do sistema.
- Quatro últimos caracteres da chave.
- Data, horário e código HTTP da tentativa.
- URL do backend integrado.

**Nunca solicitar:** chave completa, senha do usuário, conteúdo integral das variáveis do Render ou token de sessão.